

PROJECT TRACE

Task Recognition & Activity Capture Engine

OCR

UI

Visual

Interaction

Practice Project Group 21

M.Tech in Artificial Intelligence Systems | National University of Singapore

Pattern Recognition Systems | September 2026

Agenda

1 Background & Problem Statement

2 Project Objectives

3 System Architecture

4 Feature Extraction Pipeline

5 Models and Model Training

6 Training Results

7 Dataset — CUA-Suite

8 Evaluation Pipeline & Results

9 Progress

10 Challenges & Solutions

11 Future Work & Conclusion

Background

Privileged users have deleted core systems, on purpose and by mistake

S\$918,000

Loss after a dismissed system administrator deleted 180 virtual servers at a Singapore IT firm (2023)

Sentenced to 2 years 8 months in jail

INTENTIONAL | SINGAPORE, 2023

A fired admin whose access was never revoked ran a script to delete all 180 servers of a test system, one by one.

INTENTIONAL | UNITED STATES

A former worker deleted critical boot files on company servers, crippling operations for two weeks and forcing paper records.

ACCIDENTAL | EVERYWHERE

Well-meaning admins also run destructive commands on the wrong server. Different intent, same damage.

Why this project: admin sessions can be recorded, but no one can review hours of video for risky actions.

Sources: [CNA](#) | [DataBreaches.net](#) | [TechRadar](#) | [G5 Cyber Security \(citing CSO Online\)](#)

Problem Statement

PROBLEM 1 | SECURITY

Users with privileged access can accidentally or knowingly delete core Windows and Linux files and trigger security incidents, usually found only after the damage is done.

PROBLEM 2 | KNOWLEDGE

We routinely record videos of admin activities, but turning them into documents or knowledge-base articles is manual, slow and inconsistent, so the knowledge stays locked in video.

Privileged action



Risky action



Found too late

Recorded video



Manual write-up



Stays locked in video

Both problems share one gap: what happens in a recorded session is not automatically understood.

Objective

Automatically understand what was done in server screen recordings, so risky actions are caught early and every recording becomes reusable knowledge. Transform the recognized workflows into structured, searchable SOPs, runbooks, and troubleshooting guides, with traceability to support security, compliance, and proactive incident prevention.

1

Identify

Analyse recorded sessions with a tiered pipeline (frame features, sequence model, classifier) to recognise which action was performed.

2

Explain

Generate a plain-language summary of the actions so reviewers do not have to watch the full video.

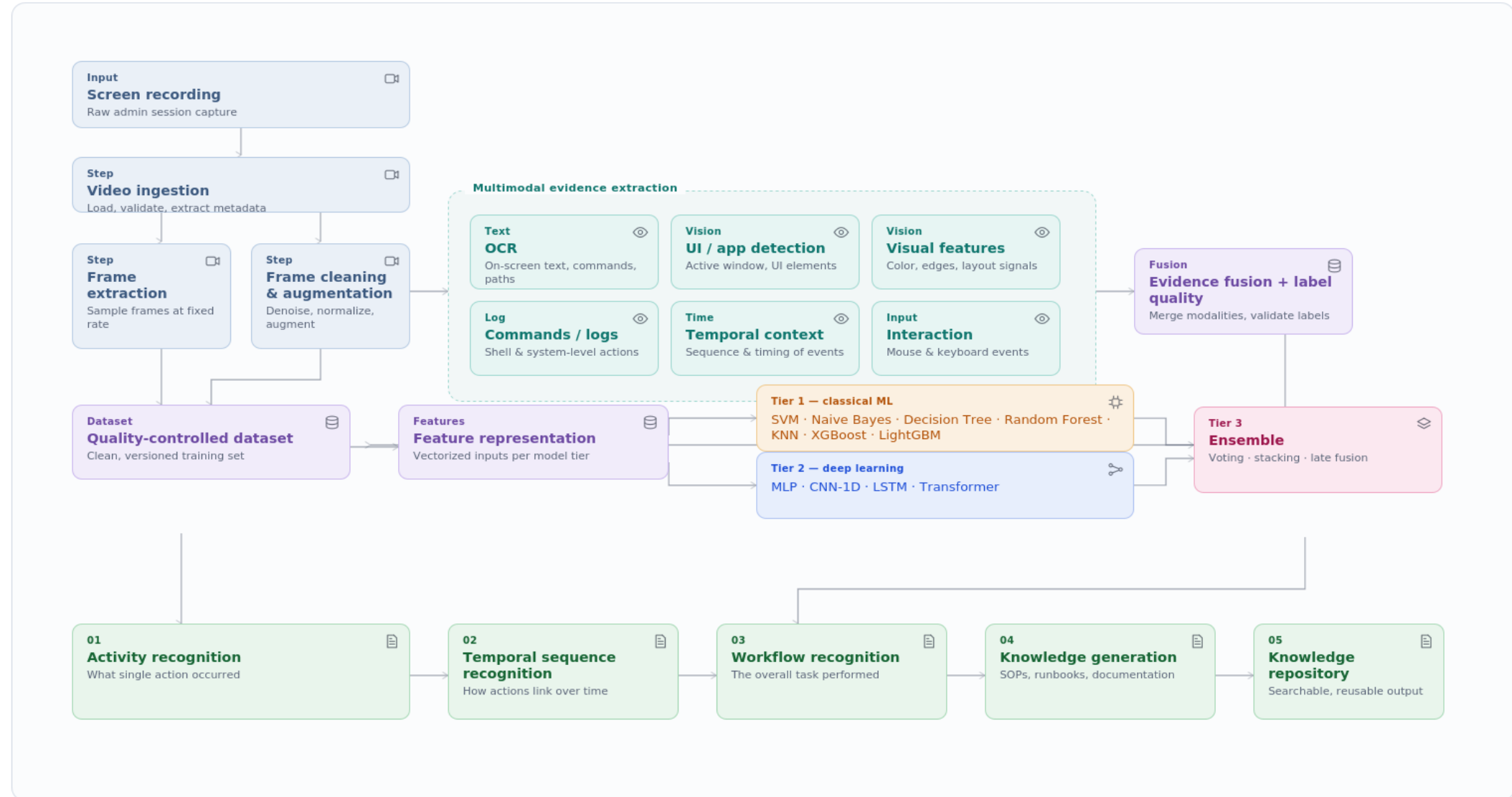
3

Prevent

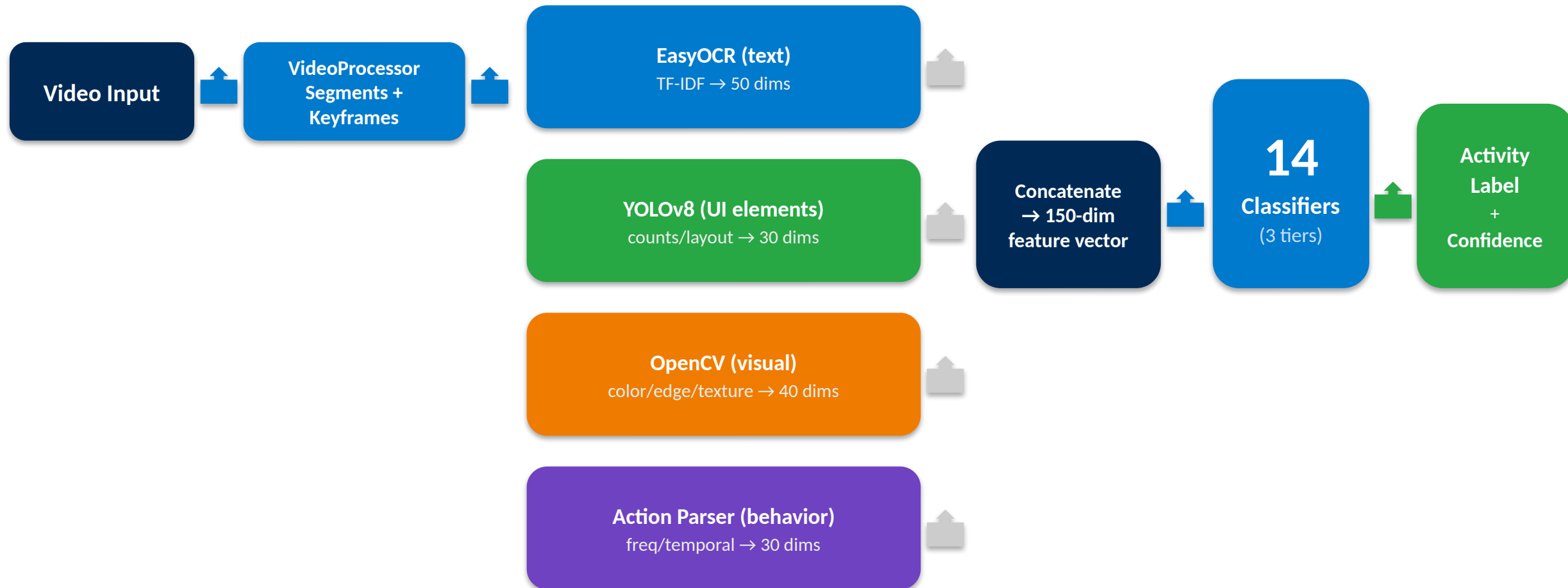
Flag high-risk actions such as deleting core files early, across many admins and organisations, not only after an incident.

TRACE Architecture

Pipeline flow: from raw screen recording to structured, searchable operational knowledge.



Multimodal Feature Extraction Pipeline



Each video segment is independently processed through all 4 extractors in parallel, producing a unified 150-dimensional feature vector

Classification Models — 14 Models, 3 Tiers



Tier 1

Classical ML (7 models)

- SVM
- Naive Bayes
- Decision Tree
- Random Forest
- KNN
- XGBoost
- LightGBM



Tier 2

Sequential Layer

- LSTM
- Transformer



Tier 3

Ensemble (3 models)

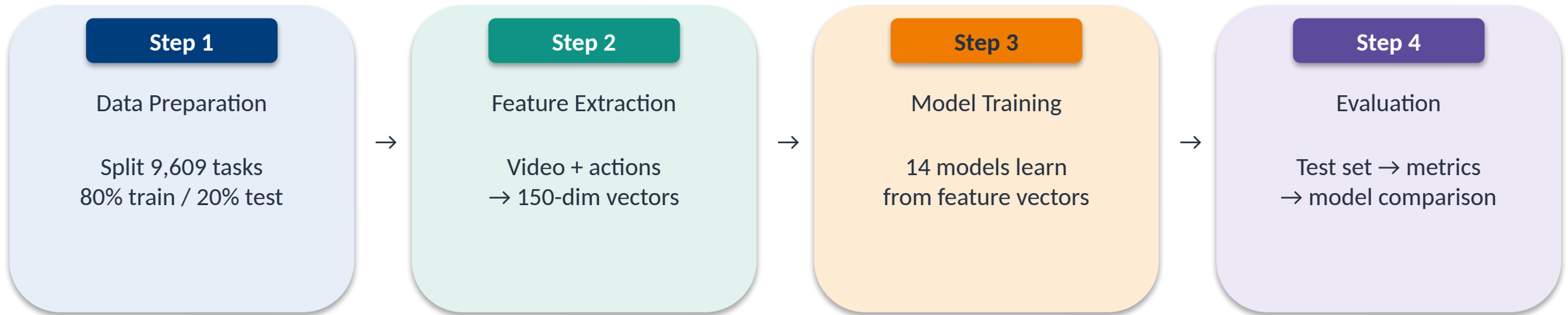
- **Voting**
Soft voting: SVM + RF + MLP
- Boosting
- Late Fusion

All 14 models run on the same 150-dim feature vector and produce comparable predictions with confidence scores

Model Training

Training Overview — 4 Steps

End-to-end pipeline from raw dataset to evaluated classifiers

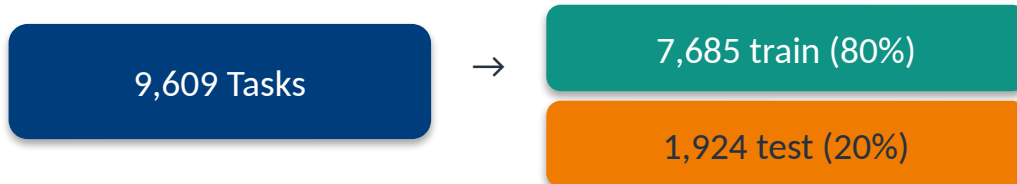


```
$ python -m app.train --dataset-root ./dataset --model-dir ./models
```

One command trains all 14 models, evaluates on held-out test set, and logs results to MLflow

Step 1 — Data Preparation

Stratified Train/Test Split



Label Distribution

Label	Count	Pct	
other	5,441	57%	
coding_editing	3,240	34%	
documentation	449	4.7%	
docker_workflow	299	3.1%	
git_operations	87	0.9%	
aws_console	45	0.5%	
debugging	38	0.4%	
kubernetes_ops	9	0.1%	
jenkins_ci_cd	1	0.01%	

Stratified Split

Same label proportions in both train and test
Ensures test set is representative of real data
`sklearn.model_selection.train_test_split(stratify=y)`

Class Imbalance Problem

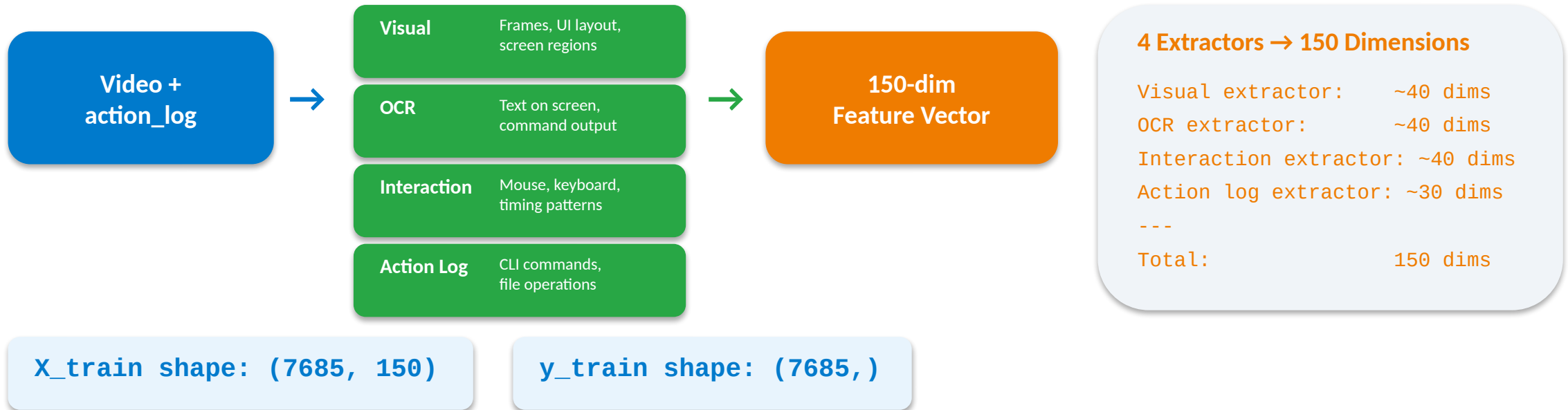
"other" dominates at 57% — model can just predict "other"
jenkins_ci_cd has only 1 sample
Data augmentation applied when < 100 training samples (5x)

Data Augmentation

Classes with < 100 samples get 5x augmentation
Adds noise to feature vectors to create synthetic samples

Step 2 — Feature Extraction

Converting raw video + actions into numeric vectors



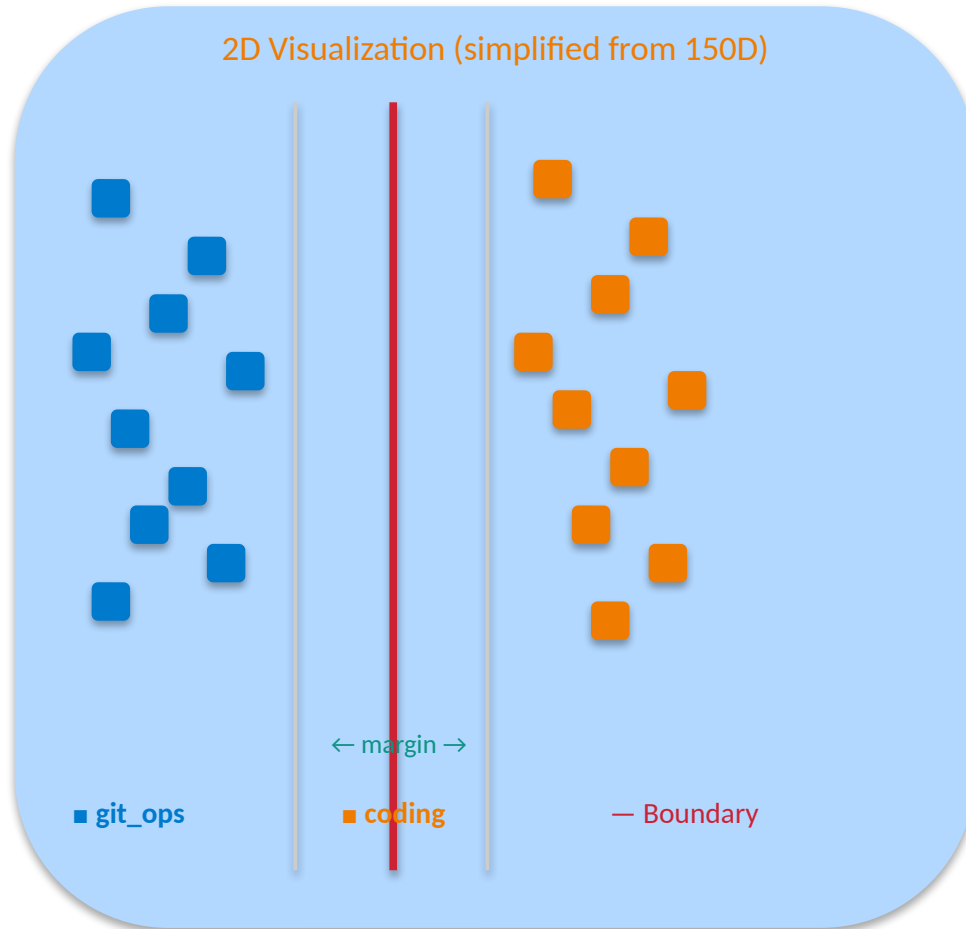
```
X[0] = [0.82, 0.75, 0.03, 0.91, ..., 0.29] # 150 features
y[0] = 0 # git_operations
```

Each row is one task, each column is one feature

This matrix is what every model trains on

Step 3 — SVM: Finding the Optimal Boundary

Support Vector Machine with RBF kernel



Maximize Margin

Finds the hyperplane that maximizes the gap between classes

RBF Kernel

$K(x, x') = \exp(-\gamma ||x - x'||^2)$ — maps to higher dimensions for non-linear boundaries

Support Vectors

The boundary points closest to the hyperplane that define the decision

Multi-class

150-dim space → find separating hyperplanes for all 9 classes (one-vs-one)

Training time: 52.6 seconds

Step 3b — Decision Tree & Random Forest

Greedy splitting and ensemble aggregation

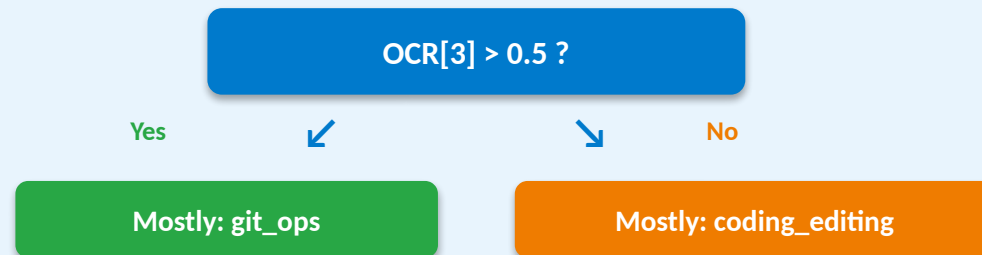
Decision Tree

Greedy recursive splitting

At each node: pick feature + threshold that maximizes

Information Gain

$$IG = \text{Entropy}(\text{parent}) - \sum (\text{weighted} \times \text{Entropy}(\text{children}))$$



Training: 0.2 seconds

- + Interpretable: you can read the rules
- + Fastest inference (2ms)
- Prone to overfitting on training data

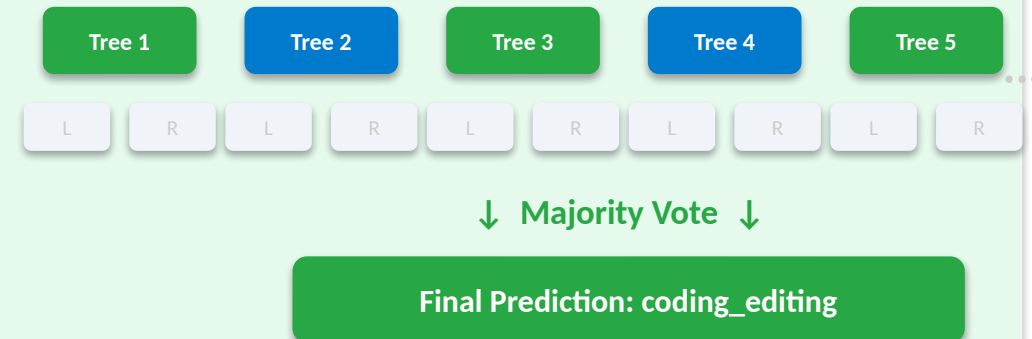
Random Forest

100 trees, each trained on random bootstrap sample

Each split considers only $\sqrt{150} \approx 12$ random features

Prediction = majority vote of 100 trees

Diversity + aggregation = robustness



Training: 0.8 seconds

- + Reduces overfitting via bagging (bootstrap aggregation)
- + Handles high-dimensional data well
- Less interpretable than a single tree

Step 3c — XGBoost: Sequential Error Correction

Gradient Boosted Decision Trees

Boosting Process: Each tree corrects the previous tree's mistakes



Key Formula

$$F_m(x) = F_{m-1}(x) + \eta \times h_m(x)$$

Training: 3.7 seconds

Key Parameters

$\eta = 0.1$ (learning rate) — small steps prevent overfitting
 Regularization $\Omega(h)$ penalizes complex trees
 100 boosting rounds total

Step 3d — MLP: Forward Pass + Backpropagation

Multi-Layer Perceptron (feed-forward neural network)

Input
150

Hidden 1
256 neurons

Hidden 2
128 neurons

Output
9 classes

$h_1 = \text{ReLU}(W_1 \cdot x + b_1)$ # 256 neurons
 $h_2 = \text{ReLU}(W_2 \cdot h_1 + b_2)$ # 128 neurons
 $\hat{y} = \text{Softmax}(W_3 \cdot h_2 + b_3)$ # 9 class probs

CrossEntropy = $-\log(\text{predicted prob of true class})$

1. Compute gradient $\partial \text{Loss} / \partial W$ for each layer
2. Update: $W = W - \text{learning_rate} \times \text{gradient}$

Network Statistics

Total parameters:	72,448
Layer 1 (150→256):	$150 \times 256 + 256 = 38,656$
Layer 2 (256→128):	$256 \times 128 + 128 = 32,896$
Layer 3 (128→9):	$128 \times 9 + 9 = 1,161$

Training Process

7,685 samples $\times$ 50 epochs = 384,250 weight updates
 Training time: 14 seconds

Step 3e — LSTM: Learning Sequences with Memory

Long Short-Term Memory recurrent neural network

Input Reshape

(150,) flat



(10 timesteps, 15 features)

LSTM Cell: 4 Gates Control Information Flow

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t])$$

What to forget from memory

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$$

What new info to store

Cell Update

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t])$$

New candidate information

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t])$$

What to output

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \# \text{ forget old + add new}$$

After 10 steps: final hidden state $\rightarrow$ Linear(64, 9) $\rightarrow$ prediction

Backpropagation Through Time (BPTT) — gradients flow back through all 10 timesteps

Training: 16 seconds

Step 3f — Transformer: Self-Attention

Learning which feature pairs matter for each class

Self-Attention Mechanism

Query

$$Q_i = W_Q \cdot x_i$$

"What am I looking for?"

Key

$$K_j = W_K \cdot x_j$$

"What do I contain?"

Value

$$V_j = W_V \cdot x_j$$

"What do I offer?"

Attention Computation

Attention weights: $\alpha = \text{softmax}(Q \cdot K^T / \sqrt{d_k})$
 Output: $z = \alpha \cdot V$ # weighted combination

Training: 60 seconds

2 attention heads, 2 encoder layers

Example: How Attention Helps Classification

OCR feature "git" attends to Interaction feature "typing commands" → strong signal for git_operations

Learns WHICH feature pairs matter for each class — no manual feature engineering needed

Step 3g — Stacking: Learning Which Model to Trust

Two-phase meta-learning ensemble

Phase A: Train Base Models on X_{train}

SVM

Random Forest

MLP

Phase B: Each base model predicts on X_{train} → probability outputs

SVM → (7685, 9)

RF → (7685, 9)

MLP → (7685, 9)

→ Stack

meta_X
(7685, 27)

Phase C: Train LogisticRegression on meta_X

Meta-learner discovers which model to trust for each class:

```
git_operations:    trust RF (0.6) > MLP (0.3) > SVM (0.1)
coding_editing:   trust SVM (0.5) > RF (0.3) > MLP (0.2)
documentation:    trust MLP (0.5) > SVM (0.3) > RF (0.2)
```

Stacking learns to combine strengths of different model families

Training: 90 seconds

Step 4 — How Models Are Evaluated

Testing on held-out data with standardized metrics



Metrics Explained

Accuracy

$\text{correct} / \text{total}$

Simple but misleading with imbalanced classes

F1 per Class

$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Balances false positives and false negatives

F1 Macro

Average of all 9 class F1 scores

PRIMARY METRIC — treats all classes equally

Why F1 Macro, not Accuracy?

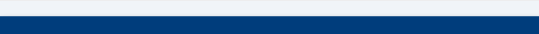
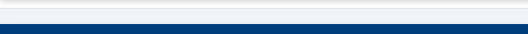
A model always predicting "other" gets 57% accuracy but F1 Macro ≈ 0.06

F1 Macro treats all 9 classes equally — minority classes matter

Latency measured with `time.perf_counter()` around `predict()`

Training Results — 14 Models Compared

Sorted by F1 Macro (primary metric)

#	Model	Tier	Accuracy	F1 Macro	Latency	
1	stacking	tier3	0.566	0.189	6951ms	
2	late_fusion	tier3	0.568	0.186	2551ms	
3	knn	tier1	0.518	0.165	344ms	
4	decision_tree	tier1	0.490	0.161	2ms	
5	random_forest	tier1	0.587	0.160	103ms	
6	xgboost	tier1	0.593	0.158	59ms	
7	naive_bayes	tier1	0.295	0.152	9ms	
8	mlp	tier2	0.570	0.148	12ms	
9	svm	tier1	0.558	0.146	1392ms	
10	transformer	tier2	0.560	0.142	18ms	
11	lstm	tier2	0.535	0.140	15ms	
12	lightgbm	tier1	0.574	0.137	37ms	
13	cnn	tier2	0.530	0.131	10ms	
14	logistic_reg	tier1	0.549	0.128	4ms	

-  Ensembles achieve best F1 — combining models reduces errors
-  XGBoost has best raw accuracy but lower F1 (biased toward majority class)
-  Decision Tree is fastest (2ms) — good for real-time
-  Deep learning didn't outperform classical ML with engineered features

Trained Models → .pkl Files

What gets saved after training completes

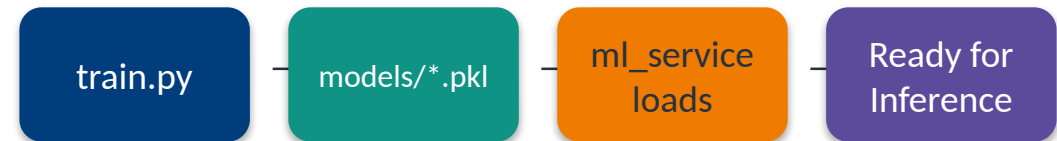
models/ directory

File	Size	Contents
svm.pkl	~2.1 MB	Support vectors + kernel params
naive_bayes.pkl	~15 KB	Class priors + likelihoods
decision_tree.pkl	~180 KB	Tree structure + thresholds
random_forest.pkl	~12 MB	100 tree structures
knn.pkl	~8.5 MB	Full training data (lazy learner)
xgboost.pkl	~1.2 MB	100 boosted trees
lightgbm.pkl	~900 KB	Gradient boosted trees
logistic_reg.pkl	~12 KB	Weight matrix (150 x 9)
mlp.pkl	~350 KB	72,448 learned weights
lstm.pkl	~500 KB	Gate weights + cell states
cnn.pkl	~280 KB	Conv filters + FC weights
transformer.pkl	~600 KB	Attention weights + FFN
stacking.pkl	~15 MB	Base models + meta-learner
late_fusion.pkl	~10 MB	Base models + fusion weights

Each .pkl contains the learned weights/parameters

Backend loads these on startup — no retraining needed.
Inference uses the saved model directly.

End-to-End Flow



What's Inside a .pkl?

Serialized with Python's pickle / joblib

Contains:

- Model architecture & hyperparameters
- Trained weights / learned parameters
- Feature scaler (if applicable)
- Label encoder mapping

Load: `model = joblib.load('models/svm.pkl')`

DataSet And Evaluation

Dataset — CUA-Suite (VideoCUA)

7,021
Tasks

6,991
Videos

~48 GB
Total Size

MIT
License

Source: [ServiceNow/VideoCUA · Datasets at Hugging Face](https://huggingface.co/ServiceNow/VideoCUA)

9 Activity Labels

git_operations

docker_workflow

kubernetes_ops

documentation

aws_console

jenkins_ci_cd

coding_editing

debugging

other

Labels derived from task instructions via keyword matching in dataset_loader.py

Dataset — Video Properties



8–30 s

Video duration
(approx., per video)



30 FPS

Frame rate



1920 × 1008

Frame resolution
(pixels)



7,890

Data rate (as provided
in the video metadata)



Application

Videos contain different activities performed within the same software environment.

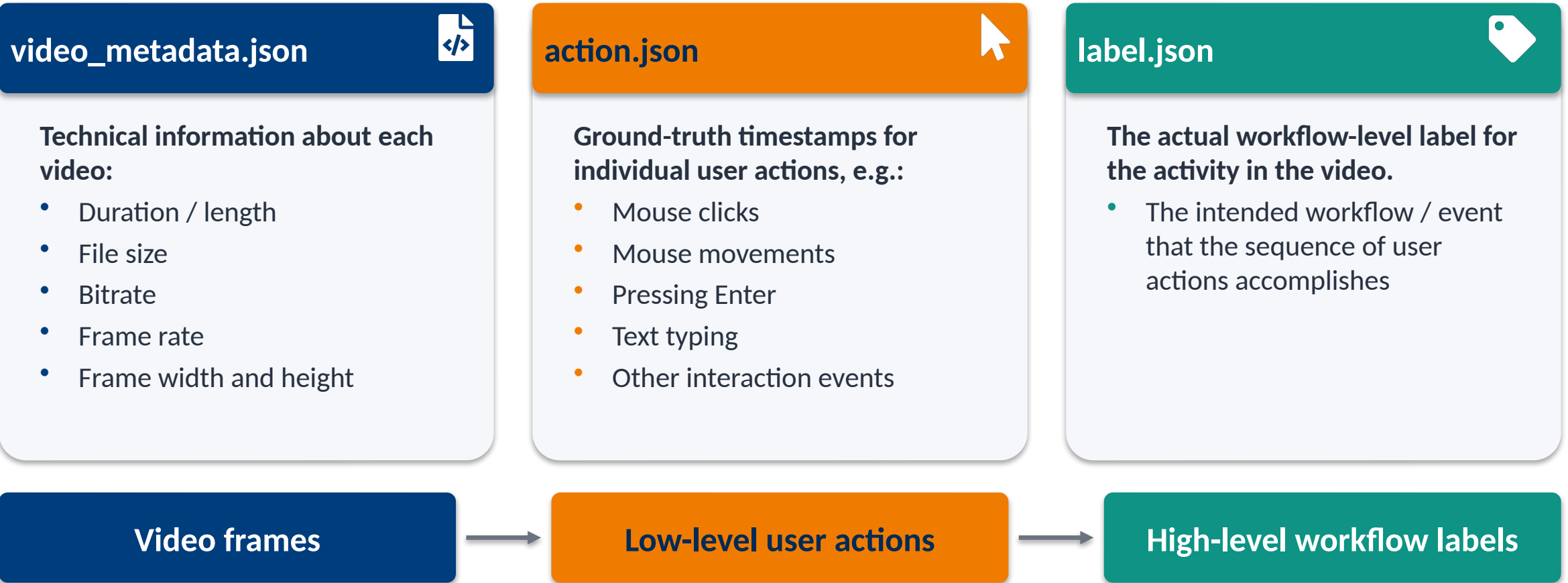


Activity

Each video represents a different workflow or sequence of user interactions.

Dataset — Annotation Files

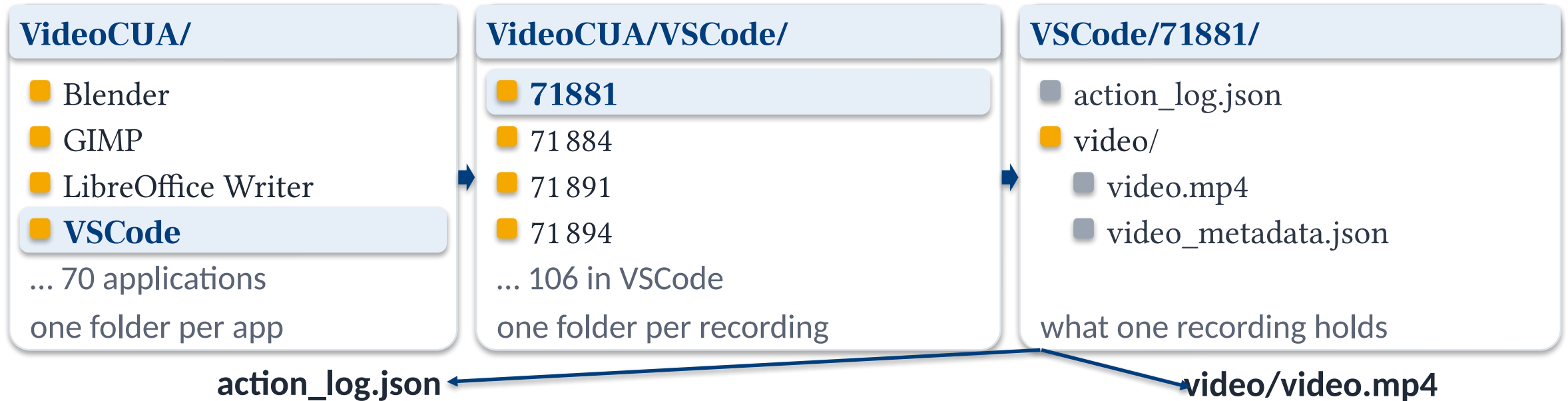
The dataset contains three main types of annotation/metadata files:



Suitable for screen activity recognition, user-action detection, and workflow understanding.

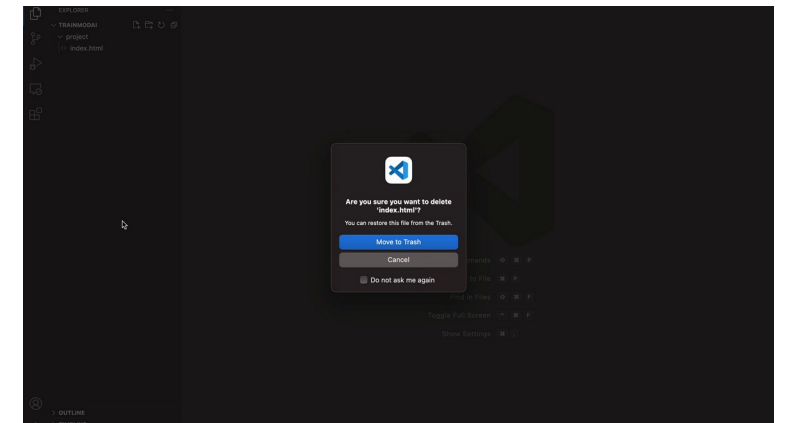
Dataset Structure

Three levels on disk — 70 applications, 6,991 recordings, a video and its logs.



```

task_instruction
"Delete the index.html file"
action_log
  1.0s MOVE_TO (327, 265)
  1.0s CLICK right
  ... 6 actions in all
  
```



Labels are derived from the task instruction by keyword matching — the same 10 classes.

Evaluation Pipeline

Five-fold cross-validation over all 6,991 recordings.

1 Split into 5 folds

The 6,991 rows go into 5 folds, grouped so a recording is never split in two.

2 Run 5 rounds — every fold is the test set exactly once

Round 1	TEST	train	train	train	train
Round 2	train	TEST	train	train	train
Round 3	train	train	TEST	train	train
Round 4	train	train	train	TEST	train
Round 5	train	train	train	train	TEST

fit	vocabulary and scaler, from 4 training folds
train	the classifier on those rows
transform	the held-out fold through the same ones
predict	1,398 recordings it has never seen

3 All 6,991 recordings predicted exactly once — scored together as one macro-F1.

Evaluation — Full Dataset

9 activity classes · 6,991 recordings · 5-fold cross-validation

0.2629

macro-F1, Stacking

3.3×

over the majority baseline

6,991

recordings, 70 apps

Top 4 — full table on the next slide

Model	Tier	macro-F1	Accuracy
Stacking	3	0.2629	0.7400
XGBoost	1	0.2498	0.7452
LightGBM	1	0.2449	0.7432
MLP	2	0.2400	0.7344
Majority baseline	—	0.0792	0.5533

How to read these numbers

- Averaged over all nine classes, not just the eight present
- Folds grouped by recording, so no video sits on both sides
- TF-IDF, scaler and encoder fitted inside the training fold only

Challenges

Data and Infrastructure Challenges

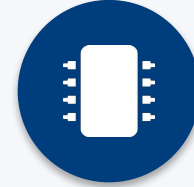
Challenge	Impact	Solution
Large Dataset (48 GB)	Cannot fit in memory or train locally	Streaming dataset loader; AWS EC2 for cloud training; Google Colab as free alternative
Train/Test Leakage	Frames from one recording landing in both splits	Task-grouped 5-fold CV; TF-IDF, scaler and label encoder fitted inside each training fold; leakage asserted by unit tests
Class Imbalance	Uneven activity label distribution	Stratified train/test split; 5x augmentation for datasets < 100 samples; F1-macro as primary metric
Different Processing Techniques for diff models	Not all models need same preprocessing	Every model has different preprocessing steps

Future Work



Training and education

Turn demonstrations into searchable tutorials, step-by-step instructions, or feedback on how a task was performed.



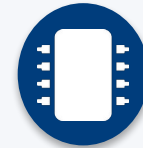
Automatic documentation

Generate a draft runbook, incident timeline, or change summary from the recognized workflow



Context-aware assistance

When the system recognizes a likely error or repeated troubleshooting pattern, it could surface relevant documentation, a prior solution, or a suggested next step.



Real Time Activity Analysis

Process screen frames and interaction events incrementally, identify activities such as editing code, running tests, debugging, or deploying, and update predictions as new evidence arrives.

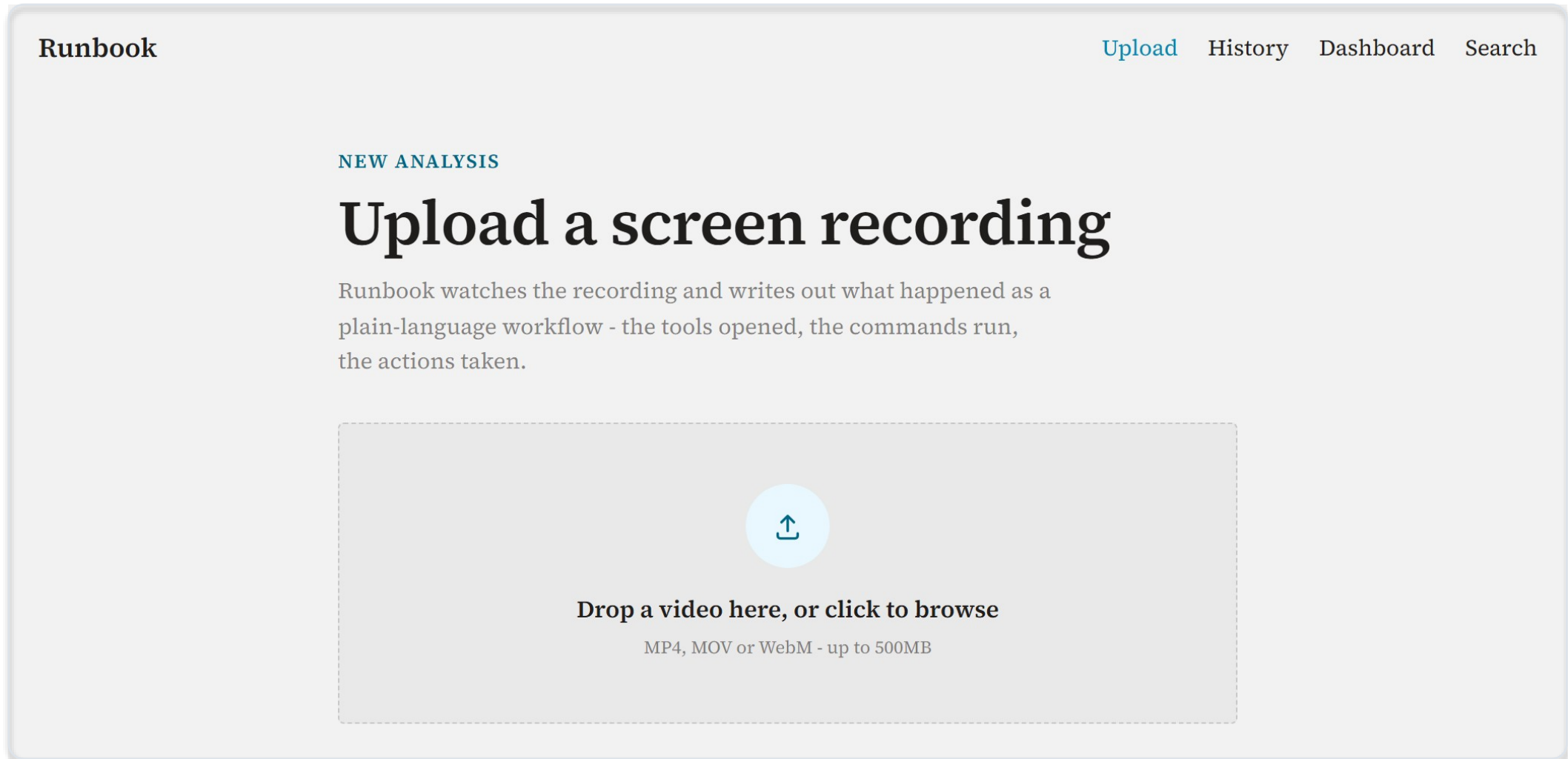


Compliance and audit

Create reviewable timelines of operational actions, subject to consent, access controls, and appropriate redaction

Upload

Drop a screen recording. Nothing else is asked for.



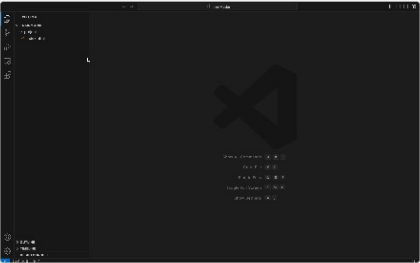
The application running on feat/frontend; the analysis is its built-in mock, so figures are placeholders.

Analyze

Frames, on-screen text and the action log are read.

Runbook

UploadHistoryDashboardSearch



Analyzing your recording

delete-file-vscode.mp4

42% - reading interface actions

Cancel

Classify

An activity label, with its confidence and the runners-up.

Runbook

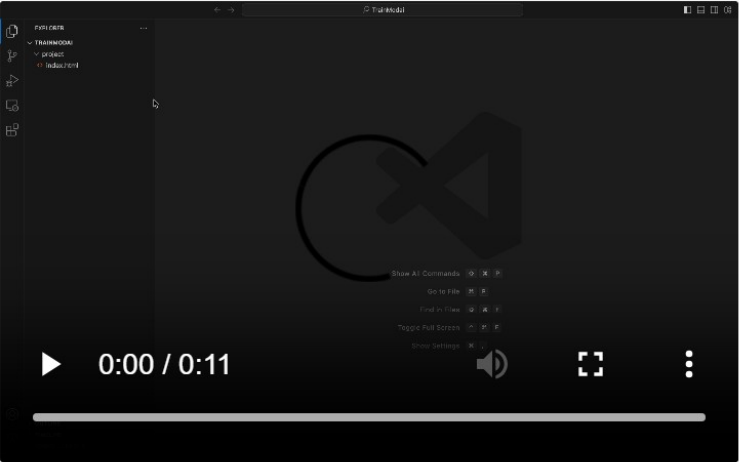
UploadHistoryDashboardSearch

CLASSIFICATION COMPLETE

Analyze another video

delete-file-vscode.mp4

Today · 1:47



coding_editing

95% confidence

Class probabilities

coding_editing

95%

other

5%

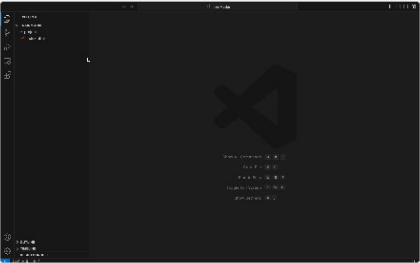
View full report (SOP)

Analyze

Frames, on-screen text and the action log are read.

Runbook

UploadHistoryDashboardSearch



Analyzing your recording

delete-file-vscode.mp4

42% - reading interface actions

Cancel

Classify

An activity label, with its confidence and the runners-up.

Runbook

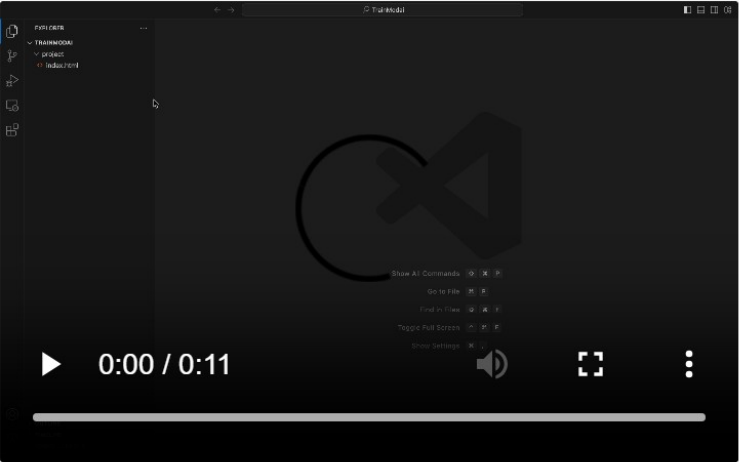
UploadHistoryDashboardSearch

CLASSIFICATION COMPLETE

Analyze another video

delete-file-vscode.mp4

Today · 1:47



coding_editing95% confidence

Class probabilities

coding_editing95%

other5%

View full report (SOP)

The SOP report

The recording becomes a numbered procedure.

Runbook

UploadHistoryDashboardSearch

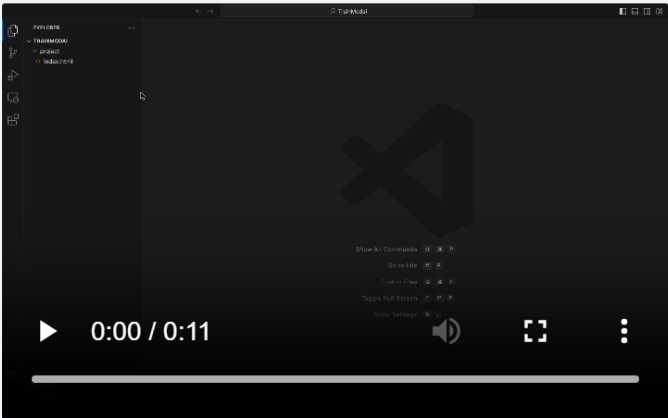
SOP REPORT

Back to results

delete-file-vscode.mp4

Today • 1:47 • 7 steps

The recording shows a developer pulling the latest changes, installing dependencies, and verifying the app in the browser.



1**Opened the code editor**0:00-0:07

The project folder loads in the editor with the file tree visible in the sidebar.

2**Opened the integrated terminal**0:07-0:16

A terminal panel opens at the bottom of the editor window.

3**Ran a shell command**0:16-0:29

"git pull origin main" is typed and run, pulling the latest changes.

Seven steps in all, each carrying the timestamps it was read from.

History

Every recording analysed so far, with the label it was given.

Runbook

Upload **History** Dashboard Search

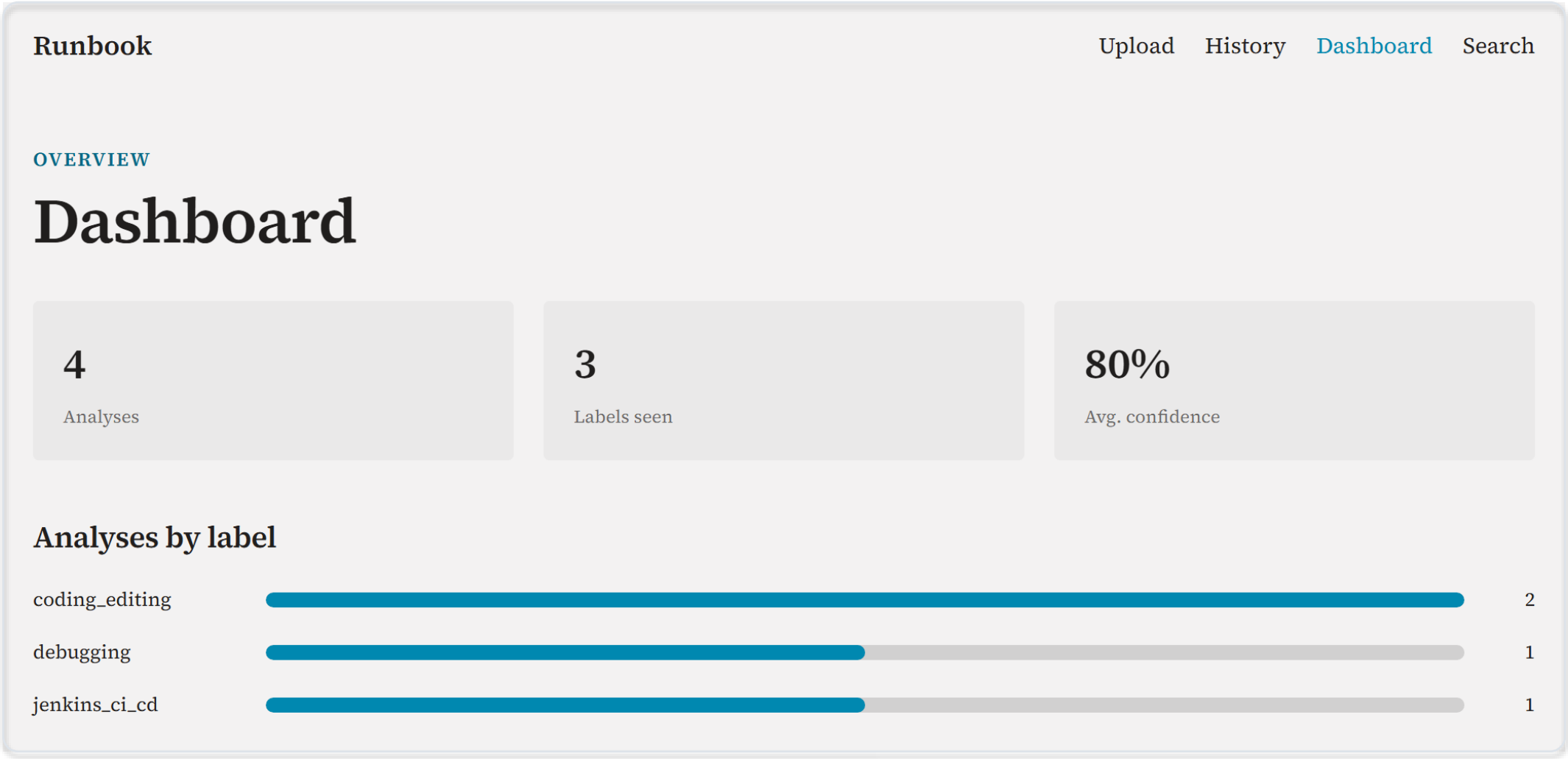
PAST ANALYSES

History

FILE	ANALYZED	DURATION	LABEL	CONFIDENCE	STATUS	
delete-file-vscode.mp4	Today	1:47	coding_editing	95%	Complete	View Report
onboarding-demo.mov	Aug 5, 2026	2:14	coding_editing	82%	Complete	View Report
bug-repro.mp4	Aug 3, 2026	0:58	debugging	74%	Complete	View Report
deploy-walkthrough.webm	Jul 29, 2026	3:02	jenkins_ci_cd	68%	Complete	View Report

Dashboard

How many recordings, which labels, average confidence.



Thank You

Group Members (name, Student ID):

1. Lakshmi Barthwal Chamoli (A0340664E)
2. Yuan Zilin (A0283899Y)
3. Stalin Sagayaraj (A0340115X)
4. Muthaiah Muneeswaran (A0019650J)

github.com/laxmi1707/PatternRecognSystem-Video2Text

References

Full source artefacts for the background incidents (slide 1)

CNA — primary report. Singapore/NCS case, S\$918,000 loss.

<https://www.channelnewsasia.com/singapore/former-employee-hack-ncs-delete-virtual-servers-quality-testing-4402141>

DataBreaches.net — secondary coverage of the CNA report.

<https://databreaches.net/2024/06/14/insider-threat-months-after-being-fired-former-employee-accessed-companys-computer-test-system-and-deleted-servers-causing-it-to-lose-s918000/>

TechRadar Pro — secondary coverage, NCS case.

<https://www.techradar.com/pro/security/ex-employee-of-major-south-east-asian-it-firm-jailed-after-he-brings-down-its-test-servers-with-scripts-found-on-google>

CSO Online — original report, US/Pratt-Read case (site not directly fetchable; cited via G5 below).

<https://www.csoonline.com/article/2123366/former-worker-sentenced-for-wrecking-servers.html>

G5 Cyber Security — reproduces the CSO Online report, US/Pratt-Read case.

<https://blog.g5cybersecurity.com/former-worker-sentenced-for-wrecking-servers/>

Appendix

AI Tool Declaration:

Figures: TRACE system architecture and pattern-recognition landscape diagrams, created by the project team with generative-AI assistance.

The project team also used Claude Code (Opu 4.6), ChatGPT, Gemini to support project ideation, review the proposed technical approach, improve the clarity and structure of selected sections, and assist with document formatting and refinement. The project team reviewed and revised the generated suggestions and is responsible for the final project presentation, technical decisions, implementation, evaluation, content, and quality of the submitted work.